

Modern adatbázis rendszerek MSc - 11. Practice

Téma: MongoDB API C# nyelven

Mappa: NEPTUNKOD_0429

Töltse fel a GitHub rendszer aktuális mappájába a forrás fájlokat!

Fejlesztő-környezet

- Visual Studio
- Visual Studio Code

1. Feladat: Környezet előkészítése és adatimport

Mappa létrehozása terminálban:

```
1 dotnet new console -n Neptunkod_APIcsharp
2 cd Neptunkod_APIcsharp
3 code .
```

MongoDB driver telepítése:

```
1 dotnet add package MongoDB.Driver
```

Adatmodellek létrehozása

Hozz létre egy Models mappát és hozz létre benne az adatmodelleket:

Etterem.cs

```
1 using MongoDB.Bson;
2 using MongoDB.Bson.Serialization.Attributes;
3
4 namespace MongoTest.Models
5 {
6     [BsonIgnoreExtraElements]
7     public class Etterem
8     {
9         [BsonId]
10         public ObjectId Id { get; set; }
11         public required string nev { get; set; }
12         public required Cim cim { get; set; }
13         public required int csillag { get; set; }
14     }
15 }
```

Cim.cs

```
1 using MongoDB.Bson.Serialization.Attributes;
2
3 namespace MongoTest.Models
4 {
5     public class Cim
6     {
7         public required string varos { get; set; }
8         public required string utca { get; set; }
9         public int hazszam { get; set; }
10    }
11 }
```

Foszakacs.cs

```
1 using MongoDB.Bson;
2 using MongoDB.Bson.Serialization.Attributes;
3
4 public class Foszakacs
5 {
6     [BsonId]
7     public ObjectId Id { get; set; }
8     public required string nev { get; set; }
9     public int eletkor { get; set; }
10    public required List<string> vegzettseg { get; set; }
11    public required string _fkod { get; set; }
12    public required string _e_f { get; set; }
13 }
```

Főprogram létrehozása

```
1 using MongoDB.Driver;
2 using MongoTest.Models;
3
4 class Program
5 {
6     static void Main(string[] args)
7     {
8         var client = new MongoClient(
9             "mongodb+srv://asd:asd@cluster0.zewoi03.
              mongodb.net/");
10        var database = client.GetDatabase("vendeglatas");
11        var etteremCollection =
12            database.GetCollection<Etterem>("ettermek");
13        var foszakacsCollection =
```

```

14         database.GetCollection<Foszakacs>("
           foszakacsok");
15     }
16 }

```

Étterem kiírás

```

1 var ettermek = etteremCollection.Find(_ => true).ToList()
  ;
2
3 foreach(var e in ettermek)
4 {
5     Console.WriteLine("-----");
6     Console.WriteLine($"N v: {e.nev}");
7     Console.WriteLine($"V ros: {e.cim?.varos}");
8     Console.WriteLine($"Utca: {e.cim?.utca}");
9     Console.WriteLine($"H zsz m: {e.cim?.hazszam}");
10    Console.WriteLine($"Csillag: {e.csillag}");
11 }

```

Futtatás

```

1 dotnet run

```

Főszakács kiírás

```

1 var foszakacsok = foszakacsCollection.Find(_ => true).
  ToList();
2
3 foreach(var f in foszakacsok)
4 {
5     Console.WriteLine("-----");
6     Console.WriteLine($"N v: {f.nev}");
7     Console.WriteLine($" letkor : {f.eletkor}");
8     Console.WriteLine($"Fkod: {f._fkod}");
9     Console.WriteLine($"EF: {f._e_f}");
10
11    Console.WriteLine($"V gzetts g:");
12    foreach (var v in f.vegzettseg)
13    {
14        Console.WriteLine($" - " + v);
15    }
16 }

```

2. Feladat: DML

Új étterem beszúrás

```
1 var ujEtterem = new Etterem
2 {
3     nev = "Valhalla",
4     cim = new Cim
5     {
6         varos = "Ny regyh za",
7         utca = "Sas",
8         hazszam = 3
9     },
10    csillag = 5
11 };
12 etteremCollection.InsertOne(ujEtterem);
13 Console.WriteLine("Sikeres beszur s!");
```

Új főszakács beszúrása

```
1 var foszakacsCollection = database.GetCollection<
2     Foszakacs>("foszakacsok");
3
4 var ujFoszakacs = new Foszakacs
5 {
6     nev = "Heged s Lajos",
7     eletkor = 25,
8     vegzettseg = new List<string> { "Le Cordon Bleu"
9     },
10    _fkod = "f3",
11    _e_f = "e1"
12 };
13 foszakacsCollection.InsertOne(ujFoszakacs);
14 Console.WriteLine("Sikeres beszur s!");
```

Csillag módosítás

```
1 var filter = Builders<Etterem>.Filter.Eq(e => e.nev, "
2     Valhalla");
3 var update = Builders<Etterem>.Update.Set(e => e.csillag,
4     3);
5 etteremCollection.UpdateOne(filter, update);*
6 Console.WriteLine("Sikeres m dos t s!");
```

30 életkor alatti törlés

```
1 var filter = Builders<Foszakacs>.Filter.Lt(f => f.eletkor
  , 30);
2 foszakacsCollection.DeleteMany(filter);
3 Console.WriteLine("Sikeres t r l s!");
```

Műszak hozzáadása (GYAKORNOK)

```
1 var filter = Builders<Gyakornok>.Filter.Eq(g => g.nev, "
  Szil gyi Istv n");
2 var update = Builders<Gyakornok>.Update.Push(g => g.
  muszak, " jszaka ");
3 gyakornokCollection.UpdateOne(filter, update);
4 Console.WriteLine("Sikeres hozzadas!");
```

3. Feladat: Lekérdezések

Szakács részlegek szerint

```
1 var reszlegek = szakacsCollection.Find(_ => true).ToList
  ();
2 foreach (var sz in reszlegek)
3 {
4     Console.WriteLine($"{sz.nev} - {sz.reszleg}");
5 }
```

4+ csillagos éttermek

```
1 var result = etteremCollection.Find(e => e.csillag >= 4).
  ToList();
2 foreach (var e in result)
3 {
4     Console.WriteLine($"N v: {e.nev} - Csillag: {e.
      csillag}");
5 }
```

Nyíregyháza vagy 5 csillag

```
1 var filter =
2 Builders<Etterem>.Filter.Eq(e => e.cim.varos, "
  Ny regyh za") |
3 Builders<Etterem>.Filter.Eq(e => e.csillag, 5);
4
5 var result = etteremCollection.Find(filter).ToList();
6
```

```

7 foreach (var e in result)
8 {
9     Console.WriteLine($"{e.nev} - {e.cim.varos} - {e.
        csillag}");
10 }

```

25-40 éves vendégek

```

1 var result = vendegCollection.Find(v => v.eletkor >= 25
    && v.eletkor <= 40).ToList();
2
3 foreach (var v in result)
4 {
5     Console.WriteLine($"{v.nev} - {v.eletkor}");
6 }

```

4. Feladat: Aggregációs pipeline

Városonkénti átlag csillag

```

1 var result = etteremCollection.Aggregate()
2     .Group(e => e.cim.varos, g => new
3     {
4         Varos = g.Key,
5         Darab = g.Count(),
6         AtlagCsillag = g.Average(x => x.csillag)
7     })
8     .ToList();
9
10 foreach (var r in result)
11 {
12     Console.WriteLine($"{r.Varos} - db: {r.Darab} -
        tlag : {r.AtlagCsillag}");
13 }

```

Szakácsok száma éttermenként

```

1 var result = szakacsCollection.Aggregate()
2     .Group(s => s._e_sz, g => new
3     {
4         EtteremKod = g.Key,
5         Szam = g.Count()
6     })
7     .ToList();
8

```

```

9 foreach (var r in result)
10 {
11     Console.WriteLine($"{r.EtteremKod} - {r.Szam} f ");
12 }

```

Legidősebb szakács éttermenként

```

1 var result = foszakacsCollection.Aggregate()
2     .SortByDescending(s => s.eletkor)
3     .Group(s => s._fkod, g => new
4     {
5         EtteremKod = g.Key,
6         LegidosebbNev = g.First().nev,
7         Kor = g.First().eletkor
8     })
9     .ToList();
10
11 foreach (var r in result)
12 {
13     Console.WriteLine($"{r.EtteremKod} - {r.LegidosebbNev
14         } ({r.Kor})");
15 }

```

Éttermek és szakácsok összekapcsolása(Lookup)

```

1 var result = etteremCollection.Aggregate()
2     .Lookup("szakacsok", "_ekod", "_e_sz", "szakacsok")
3     .ToList();
4
5 foreach (var r in result)
6 {
7     Console.WriteLine($" tterem : {r["nev"]}");
8     var szakacsok = r["szakacsok"].AsBsonArray;
9
10    foreach (var s in szakacsok)
11    {
12        Console.WriteLine($" Szak cs: {s["nev"]}");
13    }
14 }

```

Házi Feladat

Készítsd el a fenti feladatokat a saját MongoDB adatbázisoddal is. A projekt mappájának neve legyen: Neptunkod_APIcsharpOwN.